

DI/IOC, Repository & DTO Patterns in Java Spring Boot Microservices

Dependency Injection/IOC, Repository & DTO Patterns in Java Spring Boot Microservices (0-7 Years Guide)

PART 1 — DEPENDENCY INJECTION / IOC

1. What It Is

Inversion of Control (IoC) is the principle: instead of a class creating its own dependencies (new SomeService()), control is inverted — an external container creates and hands the dependencies to the class.

Dependency Injection (DI) is the technique that implements IoC in Spring — the container “injects” the required objects (beans) into a class instead of the class instantiating them itself.

Why it matters: loose coupling, easier testing (mock dependencies), centralized object lifecycle management, easy swapping of implementations.

2. The Three Types of Injection

a) Constructor Injection (Recommended — Industry Standard)

```
@Service
public class OrderService {
    private final PaymentClient paymentClient;
    private final InventoryClient inventoryClient;

    // Spring auto-injects both dependencies via constructor
    public OrderService(PaymentClient paymentClient, InventoryClient
inventoryClient) {
        this.paymentClient = paymentClient;
        this.inventoryClient = inventoryClient;
    }
}
```

@Autowired on the constructor is **optional** if there's only one constructor (Spring 4.3+ auto-detects it).

b) Field Injection (Common But Discouraged)

```
@Service
public class OrderService {
```

```

        @Autowired
        private PaymentClient paymentClient; // injected directly into
the field
    }

```

c) Setter Injection (Rare — Used for Optional Dependencies)

```

@Service
public class OrderService {
    private NotificationClient notificationClient;

    @Autowired
    public void setNotificationClient(NotificationClient
notificationClient) {
        this.notificationClient = notificationClient;
    }
}

```

3. Why Constructor Injection Is Preferred (Frequently Asked)

Reason	Explanation
Immutability	Fields can be final, preventing reassignment after construction
Fail-fast	Missing dependency causes app startup failure, not a runtime <code>NullPointerException</code>
Testability	Easy to instantiate the class manually in unit tests without Spring/reflection: <code>new OrderService(mockPaymentClient, mockInventoryClient)</code>
No circular dependency masking	Field injection can hide circular dependencies that constructor injection exposes immediately at startup
Explicit dependencies	Constructor signature documents exactly what the class needs

4. Resolving Multiple Implementations

```

public interface NotificationSender {
    void send(String msg);
}

@Component
@Primary // used when no qualifier is specified
public class EmailSender implements NotificationSender { ... }

@Component
public class SmsSender implements NotificationSender { ... }

@Service
public class AlertService {
    private final NotificationSender sender;

    public AlertService(@Qualifier("smsSender") NotificationSender
sender) {
        this.sender = sender; // explicitly picks SmsSender bean
    }
}

```

```
}
```

5. Where DI/IoC Shows Up in Microservices

- Injecting Feign clients / RestTemplate/WebClient wrappers to call other services
- Injecting repositories into services
- Injecting Kafka KafkaTemplate / listener containers
- Injecting configuration beans (@ConfigurationProperties)
- Injecting multiple Strategy/PaymentStrategy beans as a List<T> (ties back to Factory pattern)

```
@Service
public class InventoryService {
    private final InventoryClient inventoryClient; // Feign client
    injected

    public InventoryService(InventoryClient inventoryClient) {
        this.inventoryClient = inventoryClient;
    }
}
```

6. DI/IoC — By Experience Level

0-2 Years

Expected: Know what @Autowired does, difference between field and constructor injection, basic bean lifecycle. **Common questions:** What is Dependency Injection? What is @Autowired? Field vs constructor injection — which do you use and why?

2-4 Years

Expected: Resolve ambiguous bean resolution with @Qualifier/@Primary; understand bean scopes interacting with injection; write testable services using constructor injection with mocks (Mockito). **Common questions:** How do you handle multiple implementations of the same interface? What issues can field injection cause in unit testing? Explain a circular dependency issue you've faced and how you fixed it.

4-7 Years

Expected: Diagnose and resolve circular dependencies (@Lazy, redesign to break the cycle); understand BeanPostProcessor/BeanFactoryPostProcessor and how the container builds the dependency graph; design large multi-module microservices where DI wiring choices affect startup time, testability, and modularity; know why field injection is often flagged in code reviews/SonarQube in enterprise codebases. **Common questions:** How would you resolve a circular dependency between two services without restructuring the whole codebase? Explain the Spring bean lifecycle — from instantiation to @PostConstruct to destruction. Why does field injection make certain refactors and testing harder at scale? How does IoC support the Dependency Inversion Principle (the "D" in SOLID)?

PART 2 — REPOSITORY PATTERN

1. What It Is

Repository abstracts persistence/data-access logic behind a collection-like interface, so the rest of the application doesn't need to know whether data comes from JPA, MongoDB, a REST call, or an in-memory store.

2. Spring Data JPA — The Built-In Implementation

```
public interface OrderRepository extends JpaRepository<Order, Long>
{
    List<Order> findByCustomerId(Long customerId);
    Optional<Order> findByOrderNumber(String orderNumber);
    List<Order> findByStatusAndCreatedAtAfter(String status,
LocalDateTime date);
}
```

Spring generates the implementation at runtime — no code needed beyond the interface. Method names are parsed into queries automatically (derived query methods).

3. JpaRepository VS CrudRepository VS PagingAndSortingRepository

Interface	Provides
CrudRepository	Basic CRUD: save, findById, findAll, delete, count
PagingAndSortingRepository	Adds pagination and sorting on top of CRUD
JpaRepository	Extends both + JPA-specific extras: batch operations, flush(), saveAndFlush(), getReferenceById()

```
// JpaRepository gives you everything below "for free"
Page<Order> orders = orderRepository.findAll(PageRequest.of(0, 10,
Sort.by("createdAt").descending()));
```

4. Custom Queries

```
public interface OrderRepository extends JpaRepository<Order, Long>
{
    @Query("SELECT o FROM Order o WHERE o.status = :status AND
o.amount > :minAmount")
    List<Order> findHighValueOrders(@Param("status") String status,
@Param("minAmount") double minAmount);

    @Modifying
    @Query("UPDATE Order o SET o.status = :status WHERE o.id = :id")
    void updateStatus(@Param("id") Long id, @Param("status") String
status);
}
```

5. Custom Repository Implementation (When Derived/JPQL Queries Aren't Enough)

```

public interface OrderRepositoryCustom {
    List<Order> findByComplexCriteria(OrderSearchCriteria criteria);
}

@Repository
public class OrderRepositoryImpl implements OrderRepositoryCustom {
    @PersistenceContext
    private EntityManager entityManager;

    public List<Order> findByComplexCriteria(OrderSearchCriteria
criteria) {
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        // build dynamic query using Criteria API or QueryDSL
        return List.of(); // simplified
    }
}

public interface OrderRepository extends JpaRepository<Order, Long>,
OrderRepositoryCustom { }

```

6. Where Repository Shows Up in Microservices

- Each microservice typically owns **one database and its own repositories** (database-per-service pattern)
- Repositories are injected into **Service** layer, never called directly from Controllers (layering discipline)
- Read-heavy microservices often pair Repository with a **read replica / CQRS read-side** repository separate from the write-side

```

@Service
public class OrderService {
    private final OrderRepository orderRepository; // Repository
injected via DI

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order getOrder(Long id) {
        return orderRepository.findById(id)
            .orElseThrow(() -> new OrderNotFoundException(id));
    }
}

```

7. Repository — By Experience Level

0-2 Years

Expected: Use JpaRepository/CrudRepository, write derived query methods, basic save/findById/delete. **Common questions:** Difference between CrudRepository and JpaRepository? How does Spring generate query implementations from method names?

2-4 Years

Expected: Write custom @Query (JPQL/native), use pagination/sorting, understand @Modifying/@Transactional interplay, avoid N+1 query problems with @EntityGraph/JOIN FETCH. **Common questions:** How do you handle a complex dynamic search/filter query beyond derived methods? What's the N+1 select problem and how do you fix it? Difference between JPQL and native queries — when to use each?

4-7 Years

Expected: Design custom repository implementations with Criteria API/QueryDSL for dynamic queries; make database-per-service architecture decisions; handle performance at scale (indexing strategy, read replicas, connection pool tuning with HikariCP); design CQRS read/write repository split; understand transaction boundaries across repository calls in a distributed/Saga context. **Common questions:** How would you design the data layer for a service with heavy read traffic and lighter write traffic (CQRS)? How do you avoid a single shared database becoming a bottleneck/coupling point across microservices? Walk through how you'd optimize a slow repository query in production. How do repository-level transactions interact with a Saga's compensating transactions?

PART 3 — DTO PATTERN

1. What It Is

DTO (Data Transfer Object) separates the **API contract** (what clients see) from the **internal domain/entity model** (what's persisted in the database). You never expose JPA entities directly over REST.

2. Why It Matters (Frequently Asked “Why”)

Problem if you expose Entities directly	How DTO fixes it
Lazy-loading exceptions (LazyInitializationException) serializing entity relationships	DTO only carries the fields you explicitly map
Leaking internal DB structure/columns to API consumers	DTO defines its own independent shape
Over-fetching / exposing sensitive fields (e.g., password hash)	DTO explicitly excludes sensitive fields
Tight coupling between DB schema changes and API contract	Changing a DB column doesn't break the API if DTO stays stable
Circular JSON serialization on bidirectional JPA relationships	DTO breaks the cycle by design

3. Basic Example

```
@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;
    private String customerId;
    private double amount;
    private String internalNotes; // internal-only, should never be
    exposed
    @OneToMany(mappedBy = "order")
    private List<OrderItem> items; // could cause lazy-loading
    issues if serialized directly
}

// What the API actually exposes
```

```

public record OrderResponseDto(
    Long id,
    String customerId,
    double amount,
    List<OrderItemDto> items
) { }

public record OrderItemDto(String productId, int quantity) { }

```

4. Request vs Response DTOs (Common Convention)

```

// Incoming – what the client sends
public record CreateOrderRequestDto(
    @NotNull String customerId,
    @NotEmpty List<@Valid OrderItemDto> items
) { }

// Outgoing – what the client receives
public record OrderResponseDto(
    Long id,
    String status,
    double totalAmount
) { }

```

Splitting request/response DTOs (rather than one shared DTO) avoids exposing server-generated fields (like id, status) as if they were client-settable.

5. Entity ↔ DTO Mapping — MapStruct (Industry Standard)

```

@Mapper(componentModel = "spring")
public interface OrderMapper {
    OrderResponseDto toDto(Order order);
    Order toEntity(CreateOrderRequestDto dto);
}

```

MapStruct generates the mapping implementation at compile time — no runtime reflection overhead (unlike ModelMapper), and mapping bugs surface as compile errors.

Usage:

```

@Service
public class OrderService {
    private final OrderRepository orderRepository;
    private final OrderMapper orderMapper;

    public OrderResponseDto getOrder(Long id) {
        Order order = orderRepository.findById(id).orElseThrow();
        return orderMapper.toDto(order); // entity never leaves the
service layer
    }
}

```

6. Where DTOs Show Up in Microservices

- Every REST controller request/response body
- Inter-service communication payloads (Feign client request/response models)
- Kafka event payloads (event DTOs, distinct from both API DTOs and entities)
- gRPC message definitions (.proto generated classes act as DTOs)

across service boundaries)

```
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderService orderService;

    @PostMapping
    public ResponseEntity<OrderResponseDto> create(@RequestBody
@Valid CreateOrderRequestDto request) {
        OrderResponseDto response =
orderService.createOrder(request);
        return
ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
}
```

7. DTO — By Experience Level

0-2 Years

Expected: Know why entities shouldn't be exposed directly, create basic request/response DTOs, use @RequestBody/@Valid. **Common questions:** What is a DTO and why use one instead of returning the entity directly? What does @Valid do?

2-4 Years

Expected: Use MapStruct/ModelMapper for entity-DTO conversion, design separate request vs response DTOs, handle nested DTO validation, avoid LazyInitializationException by mapping inside a transactional service method. **Common questions:** How do you convert between entity and DTO cleanly at scale (manual mapping vs MapStruct vs ModelMapper — trade-offs)? How do you handle validation of nested objects in a DTO? Why did you get a LazyInitializationException and how did you fix it?

4-7 Years

Expected: Design DTO versioning strategy for API evolution (v1/v2 DTOs) without breaking existing consumers; decide when to use a single unified DTO vs multiple purpose-specific DTOs across a large microservice API surface; design DTOs for event-driven contracts (Kafka schema evolution, backward/forward compatibility, Avro/Protobuf schemas vs plain JSON DTOs); enforce DTO/entity separation across a team via architecture tests (ArchUnit) so entities can never leak past the service layer. **Common questions:** How do you version your API DTOs without breaking existing clients? How do you manage schema evolution for event DTOs published to Kafka consumed by multiple downstream teams? How would you enforce, at the build/CI level, that no @Entity class is ever returned from a @RestController? Compare MapStruct vs manual mapping vs ModelMapper for a large codebase with 100+ DTOs — which would you pick and why?

Quick Summary Table (All Three Patterns)

Pattern	Core Purpose	Key Spring Feature
DI / IoC	Decouple object creation from usage	@Autowired, constructor injection,

Repository	Abstract persistence logic	ApplicationContext JpaRepository, CrudRepository, @Query
DTO	Separate API contract from internal domain model	Request/Response DTOs, MapStruct, @Valid

Senior-Level Integration Point (Often Asked Together)

“Walk me through the full flow of a POST /orders request in your service.” 1. @RestController receives CreateOrderRequestDto (**DTO**), validated via @Valid 2. Controller calls OrderService (**DI** — injected via constructor) 3. Service maps DTO → Entity (via MapStruct), calls OrderRepository.save() (**Repository**) 4. Service maps saved Entity → OrderResponseDto (**DTO**) before returning 5. Entity never crosses the controller boundary at any point